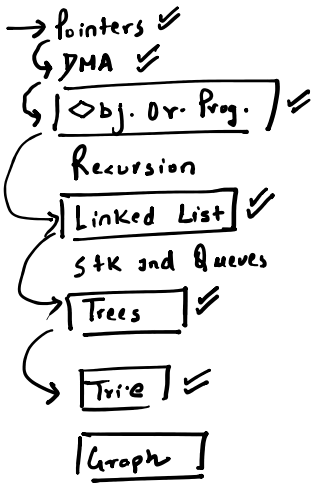
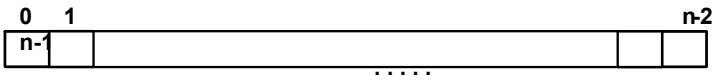
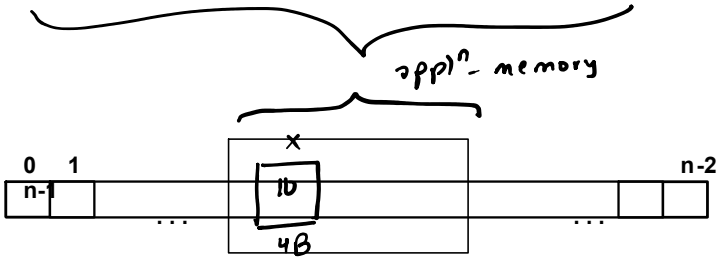


Introduction to Dynamic Memory Allocation

- We can visualize the system memory **(RAM)** as a sequence of memory cells such that each memory cell is of **1B (8 bits)** and is addressable.

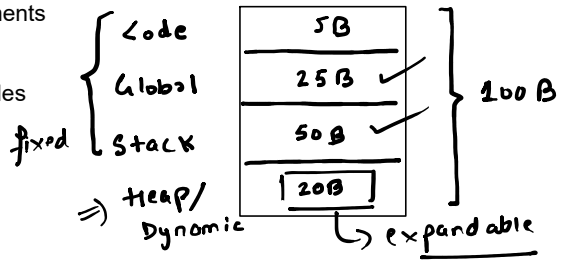


- When we run a C++ program, a portion of system memory is allocated for program execution which is known as **application memory.**




```
main() {
    int x = 10;
}
```

- The application memory is divided into **four** segments
  - Code or Text : to store program instructions
  - Global/Static : to store global and static variables
  - Stack : to store local variables
  - Heap or Dynamic

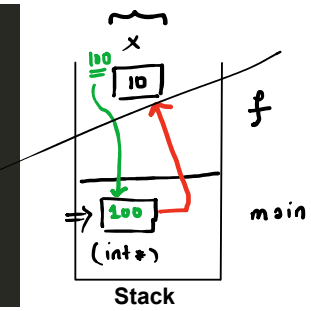


Stack Segment

- The memory allocated for stack segment of application memory is **fixed**.
- The size of the stack frame for a function must be known at compile time.
- The process of allocation and deallocation of memory is handled by **OS***.

```
int* f() {
    int x = 10;
    return &x;
}

int main() {
    int* xptr = f();
    cout << *xptr << endl;
    return 0;
}
```



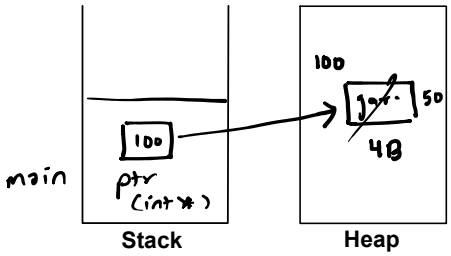
Heap or Dynamic Memory

```
type * ptr = new type;
```

- The memory allocated for the heap segment is dynamic in nature.
- The programmer handles the process memory allocation and deallocation.

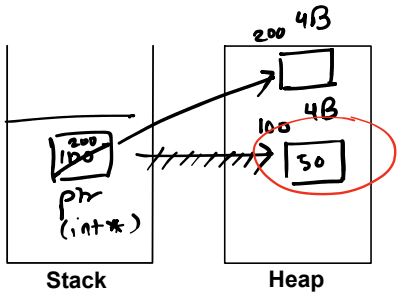
- The memory allocated for the heap segment is dynamic in nature.
- The programmer handles the process memory allocation and deallocation.

```
main() {
int * ptr = new int;
*ptr = 50;
cout << *ptr;
}
```



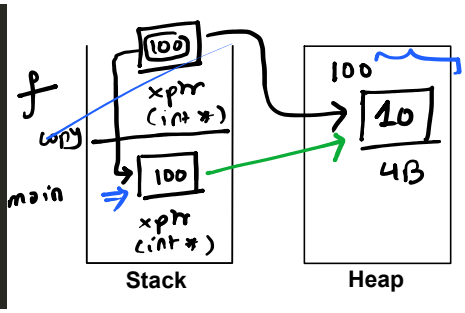
Memory Leak

```
main() {
int * ptr = new int;
*ptr = 50;
cout << *ptr; //50
delete ptr;
ptr = new int;
}
```




```
int* f() {
    int* xptr = new int;
    *xptr = 10;
    return xptr;
}

int main() {
    int* xptr = f();
    cout << *xptr; //10
}
```



1D arrays on the Stack or Static Memory

should be known at comp.-time

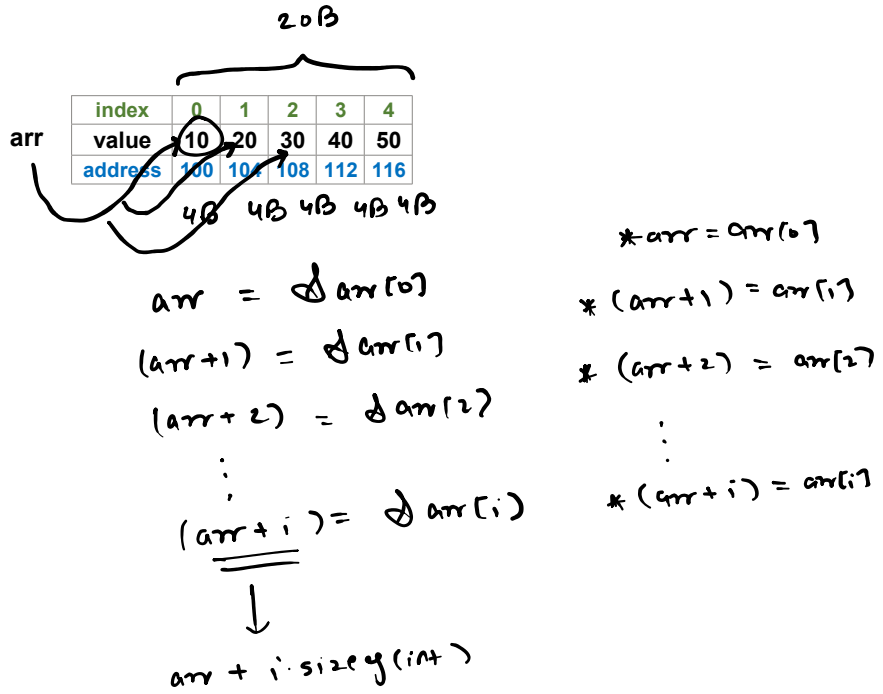
An array is a **linear** data structure, referred by a single name, which is used to store a **sequence** of values of the **same type**.

```
int arr[5];
```

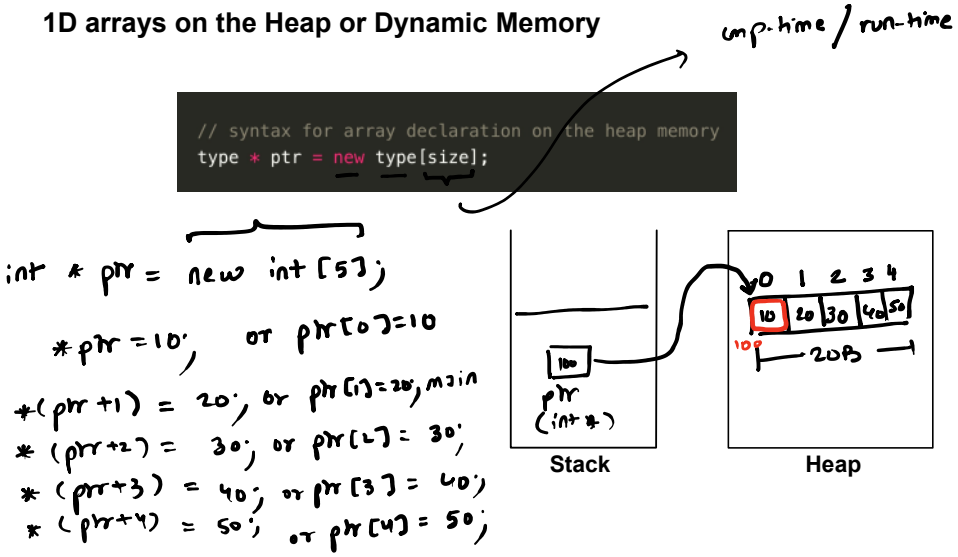
```
// syntax for array declaration

type name[size];
```

In C++, we can think of **name** of an array as a **pointer** to the element at the **0th** index.



1D arrays on the Heap or Dynamic Memory



By default, when we declare an array on the heap memory, it contains **garbage** value.

```
int* arr = new int[5]{10, 20, 30, 40, 50};
```

During the **initialization** of an array created on the heap memory, specifying the size of the array is **optional**. Also, the size of the **initializer list** should not exceed the array size.

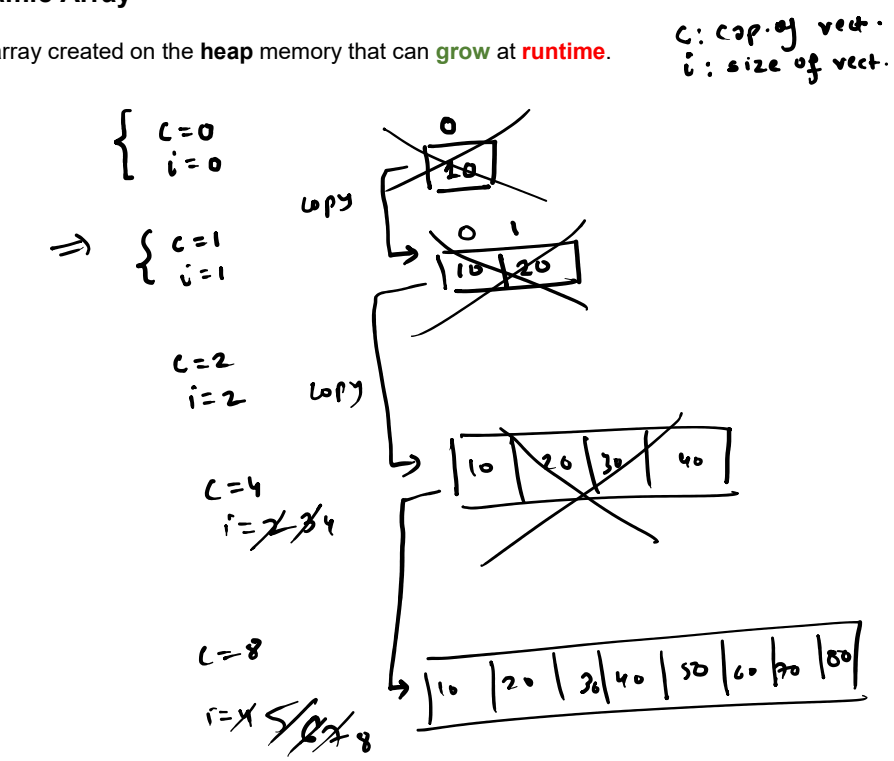
To access elements of a 1D array created on heap or dynamic memory, we can use the same syntax that is used to access elements of a 1D array created on the stack memory.

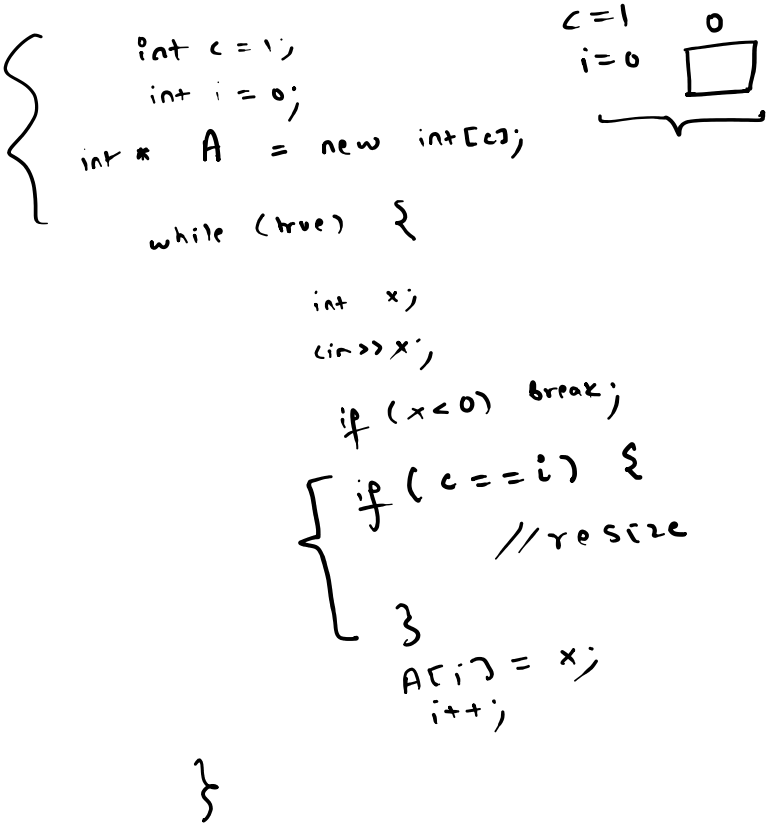
Dynamic Array

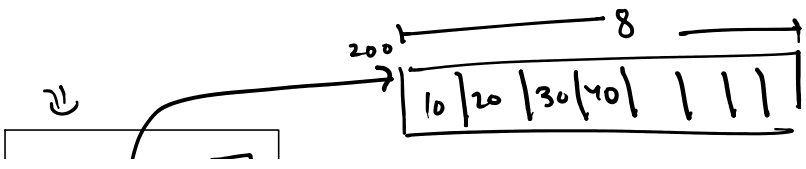
C: copy vect.

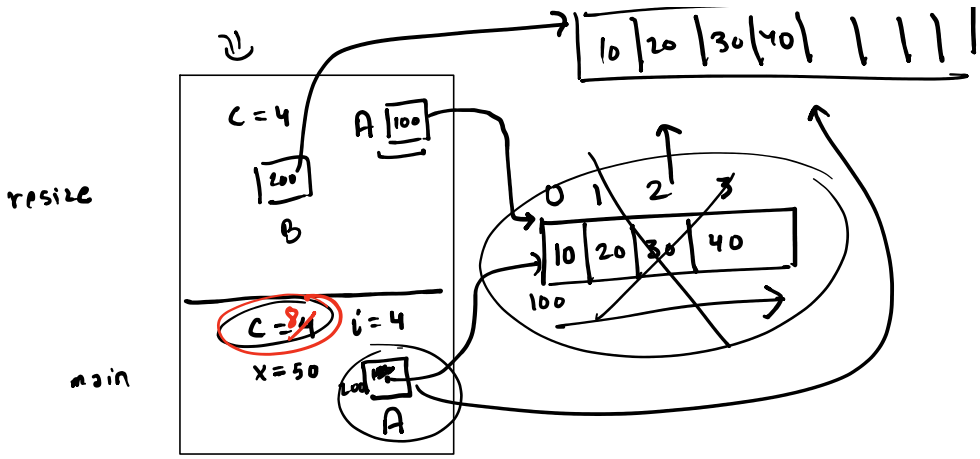
Dynamic Array

It an array created on the **heap** memory that can **grow** at **runtime**.









[nw][2]

2D arrays on the Stack or Static Memory

A 2D-array is an **array of 1D arrays**, referred by a single name, which is used to store a **sequence** of values of the **same type** and can be visualized as a **matrix**.

```
// syntax for a 2D-array declaration

type name[rows][cols];
```

arr

	0	1	2	3
0	10	15	20	25
1	30	35	40	45
2	50	55	60	65

arr

0				1				2			
0	1	2	3	0	1	2	3	0	1	2	3
10	15	20	25	30	35	40	45	50	55	60	65
100	104	108	112	116	120	124	128	132	136	140	144

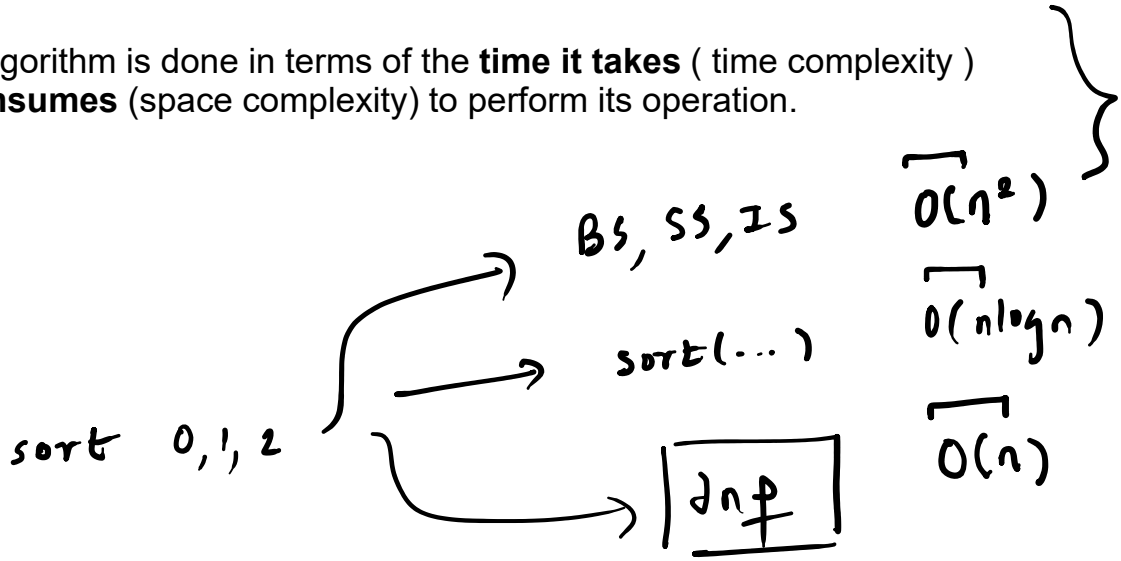
arr[0] arr[1] arr[2]

2D arrays on the Heap or Dynamic Memory

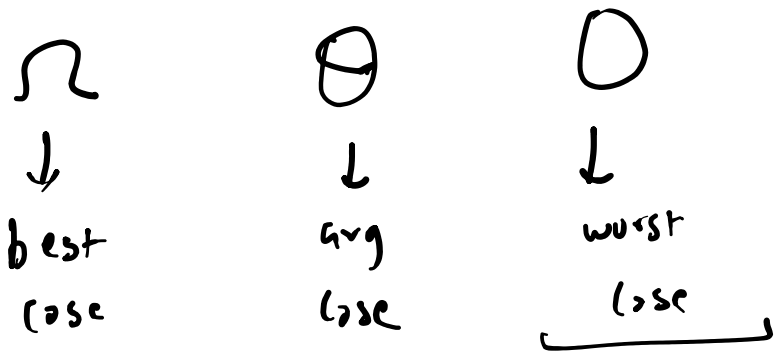
A 2D-array is an **array of 1D arrays.**

Algorithm Analysis

The analysis of an algorithm is done in terms of the **time it takes** (time complexity) and the **space it consumes** (space complexity) to perform its operation.



The analysis of an algorithm is always done in terms of the **input size**. Moreover, we mostly interest in the **worst-case analysis** and use the **Big-O notation** to report the worst-case time and space complexity of an algorithm.



To analyze the **time complexity** of an algorithm, we've to first identify algorithm type

- Iterative (contains only loops)
- recursive (contains recursive calls)
- neither iterative nor recursive - mostly take **constant** time i.e. **O(1)**

To analyze the **space complexity** of an algorithm, you've to analyze how much **extra** space or **auxiliary** space the algorithm consumes with respect to the size of the input.

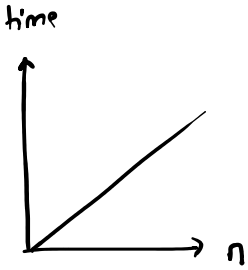
Time Complexity Analysis of Iterative Algorithms

```
void f(int n) {
    for(int i=1; i<=n; i++) {
        cout << "*" << " ";
    }
}
```

iterations = n

const ∴ n · const

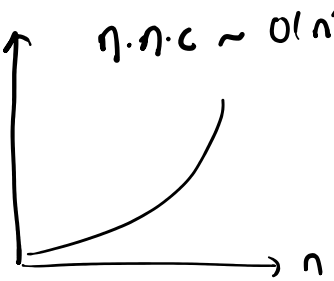
O(n)




```
void f(int n) {
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=n; j++) {
            cout << "*";
        }
    }
}
```

n · c

n · n · c ~ O(n²)




```
void f(int n) {
    int i = 1;
    int s = 1;
    while(s < n) {
        i++;
        s += i;
    }
}
```

const

~ √n · const ∴ O(√n)

i	s	
1	1	< n — ①
2	3	< n — ②
3	6	< n — ③
4	10	< n — ④
5	15	⋮
⋮	⋮	⋮
K	K(K+1)/2 ~ K²	< n — K !!, false

i² ≤ n ∴ i ≤ √n

K-1 times

K² = n

K = √n

```
void f(int n) {
    for(int i=1; i*i <= n; i++) {
        cout << "*";
    }
}
```

$\sqrt{n}$ · const $\therefore O(\sqrt{n})$

```
void f(int n) {
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=i; j++) {
            cout << "*";
        }
    }
}
```

i	j	# stars
1	1 to 1	1
2	1 to 2	2
3	1 to 3	3
⋮	⋮	⋮
n	1 to n	n

$\sum = \frac{n(n+1)}{2} \sim n^2 \cdot const$

$\therefore O(n^2)$

```
void f(int n) {
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=i*i; j++) {
            cout << "*";
        }
    }
}
```

i	j	# stars
1	1 to 1^2	1^2
2	1 to 2^2	2^2
3	1 to 3^2	3^2
⋮	⋮	⋮
n	1 to n^2	n^2

$\sum = \frac{n(n+1)(2n+1)}{6}$

$\sim n^3 \cdot const$

$O(n^3)$

```
void f(int n) {
    for(int i=1; i<n; i*=2) {
        cout << "*";
    }
}
```

i = i * 2

const

i		
$2^0 = 1$	$< n$?	①
$2^1 = 2$	$< n$?	②
$2^2 = 4$	$< n$?	③
$2^3 = 8$	$< n$?	④
$2^4 = 16$	⋮	
⋮		
2^K	$< n$?	true ⟹ K+1
	⇓ false	

$2^K = n$

$K = \log_2 n$

$\log_2 n \cdot const \therefore O(\log_2 n)$

2^k $< n$? ⟹ ... !! false stop

$\frac{n}{2} \cdot \frac{n}{2} \cdot \log n \cdot$ const

k times

$O(n^2 \log_2 n)$

```
void f(int n) {
    for(int i=n/2; i<=n; i++) {
        for(int j=1; j<=n/2; j++) {
            for(int k=1; k<n; k*=2) {
                cout << "*";
            }
        }
    }
}
```

$\frac{n}{2}$ $\frac{n}{2}$ $\log_2 n$ const

```
void f(int n) {
    for(int i=n/2; i<=n; i++) {
        for(int j=1; j<n; j*=2) {
            for(int k=1; k<n; k*=2) {
                cout << "*";
            }
        }
    }
}
```

$\log_2 n$ $\log n$ $\frac{n}{2}$

$\frac{n}{2} \log n \log n \cdot$ const

$\therefore O(n(\log n)^2)$

```
void f(int n) {
    while(n > 1) {
        n /= 2;
    }
}
```

n = n/2; const

$\log n \cdot$ const $\therefore O(\log_2 n)$

$\frac{n}{2^0} = n$ >1? ✓ -(1)

$\frac{n}{2^1} = n/2$ >1? ✓ -(2)

$\frac{n}{2^2} = n/4$ >1? ✓ -(3)

$\frac{n}{2^3} = n/8$ >1? ✓ -(4)

$\vdots$

$\frac{n}{2^k}$ >1? — (k+1)

!! false

k times

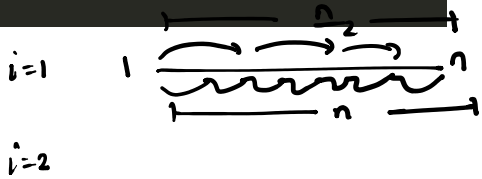
$\frac{n}{2^k} = 1$

$2^k = n$

$k = \log_2 n$

```
void f(int n) {
    for(int i=1; i<=n; i++) {
        for(int j=1; j<=n; j+=i) {
            cout << "*";
        }
    }
}
```

j = j+i



i	j
1	1 to n
2	1 to n
3	1 to n
⋮	⋮
n	1 to n

n = n/1	
n/2	
n/3	
⋮	
1 = n/n	

$$T = \frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \dots + \frac{n}{n}$$

$$= n\left(\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}\right)$$

$$O(n \cdot \log_2 n)$$

```
int f(int arr[], int n, int t) {
    int s = 0;
    int e = n-1;
    while(s <= e) {
        int m = s+(e-s)/2;
        if(arr[m] == t) {
            return m;
        } else if(arr[m] > t) {
            e = m-1;
        } else {
            s = m+1;
        }
    }
    return -1;
}
```

```
void f(int arr[], int n) {
    for(int i=1; i<n; i++) {
        for(int j=0; j<n-i; j++) {
            if(arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}
```

```
void f(int n) {
    cout << n*10;
}
```

Space Complexity Analysis of Iterative Algorithms

```
void f(int n) {
    int a = 0;
    int b = 1;

    if(n == 0 || n == 1) {
        return n;
    }

    for(int i=2; i<=n; i++) {
        int c = a+b;
        a = b;
        b = c;
    }

    return b;
}
```

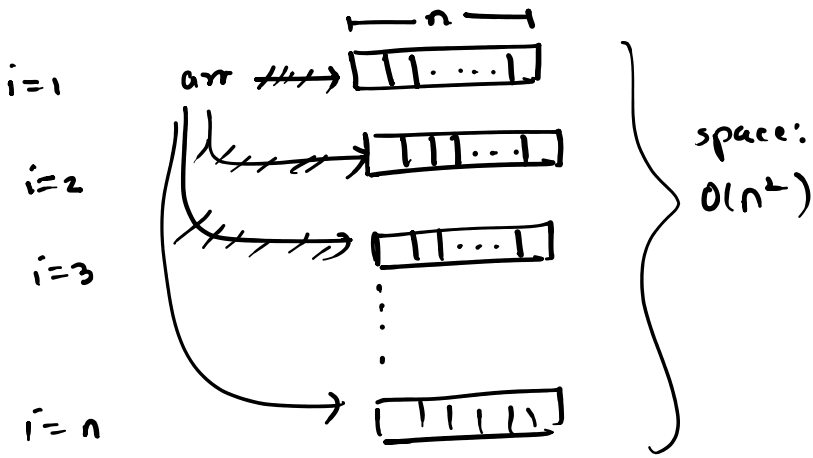
n-1 times ~ n

2 to n

time: n · const ∴ O(n)

space: O(1)

```
void f(int n) {
    for(int i=0; i<=n; i++) {
        int* arr = new int[n];
    }
}
```



```
void f(int n) {
    for(int i=0; i<=n; i++) {
        int* arr = new int[n];
        delete [] arr;
    }
}
```